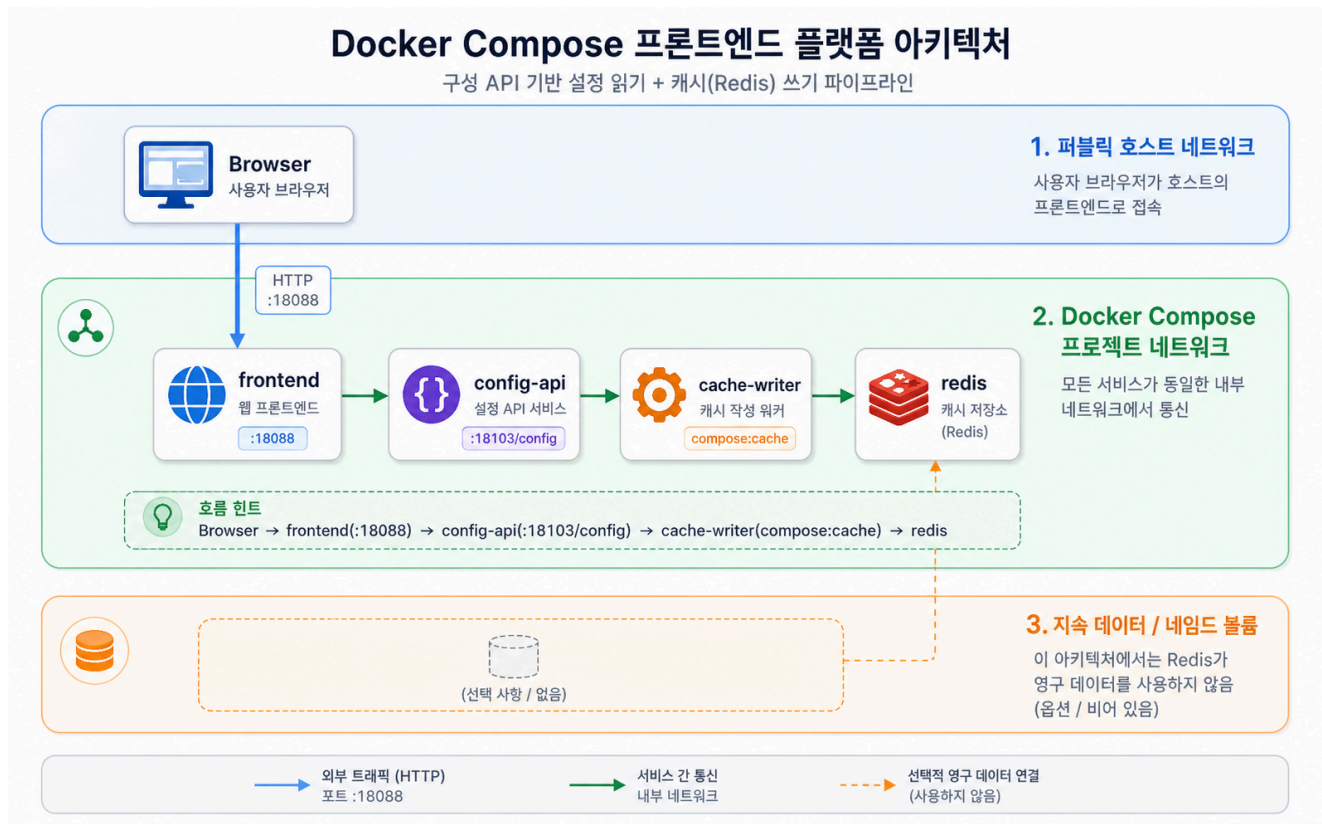


4교시: 토스형 프론트엔드 플랫폼 template



수업 목표

- frontend preview, config API, Redis cache를 함께 실행한다.
- API endpoint와 feature flag가 runtime config로 분리되는 방식을 확인한다.
- cache data lifecycle과 DB data lifecycle을 구분한다.

언제 쓰는가

W1D4의 프론트엔드 플랫폼 사례를 Compose로 줄인다. 화면은 nginx로 제공하고, 설정은 config API가 제공한다. Redis는 preview/cache/feature 실험에서 외부 backing service로 붙는다.

Template

```
cd week2/day5/labs/compose-architectures/03-web-redis
docker compose config
docker compose up -d
docker compose ps
```

구성:

Service	역할	공개 범위
web	cache template 안내 web app	host 18088
config-api	frontend runtime config API	host 18103
redis	Redis cache	Compose network 내부
cache-writer	Redis에 값을 쓰는 sample app	logs로 결과 확인
redis-cli	수동 확인 도구	--profile tool로 실행

Check

```
curl -I http://localhost:18088
curl -s http://localhost:18103/config
docker compose logs cache-writer --tail 20
docker compose exec redis redis-cli GET compose:cache
docker compose --profile tool run --rm redis-cli
```

Expected:

```
HTTP/1.1 200 OK
"newCheckout":true
hit-from-cache-writer
PONG
```

실무 해석

cache는 app container 안의 변수나 파일이 아니다. 별도 service다. 그래서 app이 재시작되어도 Redis service가 살아 있으면 cache는 유지될 수 있고, Redis container가 사라지면 cache도 사라질 수 있다.

Cleanup

```
docker compose down
```